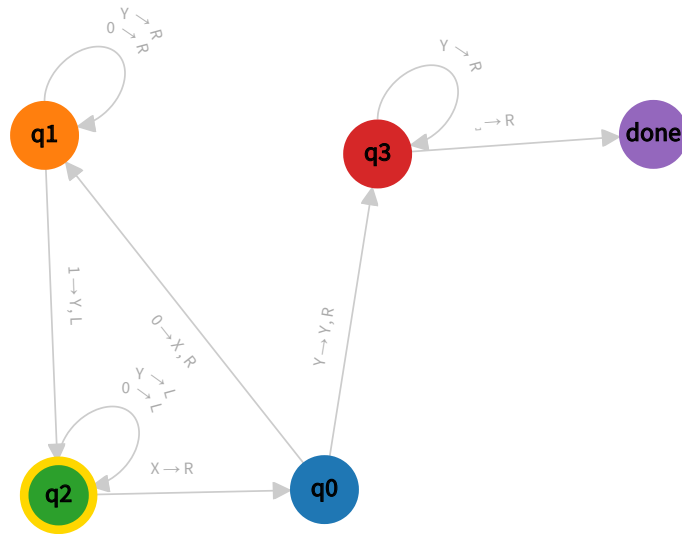




```

1 input: '0011'
2 blank: ' '
3 start state: q0
4 table:
5   q0:
6     0: {write: X, R: q1}
7     Y: {write: Y, R: q3}
8   q1:
9     0: R
10    Y: R
11    1: {write: Y, L: q2}
12   q2:
13     0: L
14     Y: L
15     X: {R: q0}
16   q3:
17     Y: R
18     ' ': {R: done}
19 done:

```

Try it out

Run the machine to see it in action. At any time, you can **step** or **pause** to get a closer look, or **reset** to start over.

There are over a dozen different example machines to explore.

Most of the examples take input. Experiment with different inputs to see what happens! Edit the code and click *Load machine* to sync your changes.

What's going on?

The colored circles are *states*. The squares underneath are *tape cells*.

The current state and tape cell are highlighted.

At each step, a Turing machine reads its current state and tape symbol, and looks them up in its *transition table* for an instruction. Each *instruction* does 3 things:

1. **write** a symbol to the current tape cell
2. **move** to the left or right by one cell
3. **set** the new state

That's it!

This repeats step after step, until the machine reaches a combination of state and symbol that don't have an instruction defined. At that point it *halts*.

► In more detail

For a very readable introduction to Turing machines, including their significance and interesting implications, see the excellent entry in the Stanford Encyclopedia of Philosophy (<http://plato.stanford.edu/entries/turing-machine>).

Make your own

Make spinoffs from the examples or your own creations by using *Edit > Duplicate document*. You can also start from scratch with a new blank document.

All it takes to describe a Turing machine is a start state, blank symbol, and transition table.

Example

```
# Adds 1 to a binary number.
input: '1011'
blank: ' '
start state: right
table:
  # scan to the rightmost digit
  right:
    1: R
    0: R
    ' ': {L: carry}
  # then carry the 1
  carry:
    1: {write: 0, L}
    0: {write: 1, L: done}
    ' ': {write: 1, L: done}
  done:
```

Here the states are `right`, `carry`, and `done`.
The symbols are `'1'`, `'0'`, and `' '`.

We designate one state the *start state*, and one symbol the *blank* symbol (found on unmarked tape cells).

A state and a symbol together map to an instruction. In the `carry` state, for instance, the symbol `1` maps to the instruction `{write: 0, L}`. When no instruction is defined, as in the `done` state, the machine halts.

Instruction format

The general form of an instruction has three parts:

```
{write: symbol, move: state}
```

- `{write: 1, L: done}` writes the symbol `1`, moves the tape head left (`L`), and goes to the `done` state.

For brevity, you can omit the symbol and state if they stay the same.

- `{L: carry}` writes the same symbol that was read, moves the tape head left, and goes to the `carry` state.
- `R` (shorthand for `{R}`) simply moves the tape head right. It writes back the same symbol and stays in the same state.

Tips

Editor keyboard shortcuts

Ctrl - S Load machine
Save changes and load the machine.

Ctrl - / Toggle comment

Tab Indent selection

Shift - Tab Unindent selection

Alt - ↑ Move line up

Alt - ↓ Move line down

Ctrl - Shift - D Duplicate line/selection

Ctrl - Alt - → Add next occurrence to selection
Like a quick “find”. Useful for renaming things.

Ctrl - F Find

Ctrl - H Find and Replace

More keyboard shortcuts are available on the full list (<https://github.com/ajaxorg/ace/wiki/Default-Keyboard-Shortcuts>).

Misc. tips

- Drag or click a state node to fix it in place; double-click to release it.
- Don't hesitate to duplicate. Use *Edit > Duplicate document* to create a snapshot of your current document whenever you're about to make a larger edit.

- Changes are autosaved to your browser's local storage. They will stay there between sessions, but be careful when clearing browser data. You can create backups by downloading your documents.
 - Incognito / Private Browsing mode uses a separate temporary storage. This is useful for importing or editing documents that you want to try but don't want to keep.
- More about the syntax



(<https://github.com/aepsilon/turing-machine-viz>)